

INTRODUCTION

This project demonstrates the use of MySQL for database management and analysis. An employee dataset was imported into MySQL Workbench and various database operations including functions, triggers, stored procedures, joins, user management, backup and restoration were performed.

DATASET

The dataset was obtained from Kaggle and contains employee information such as job title, years of experience, education level, industry, company size, location, Remote work, Status, Certification and salary.

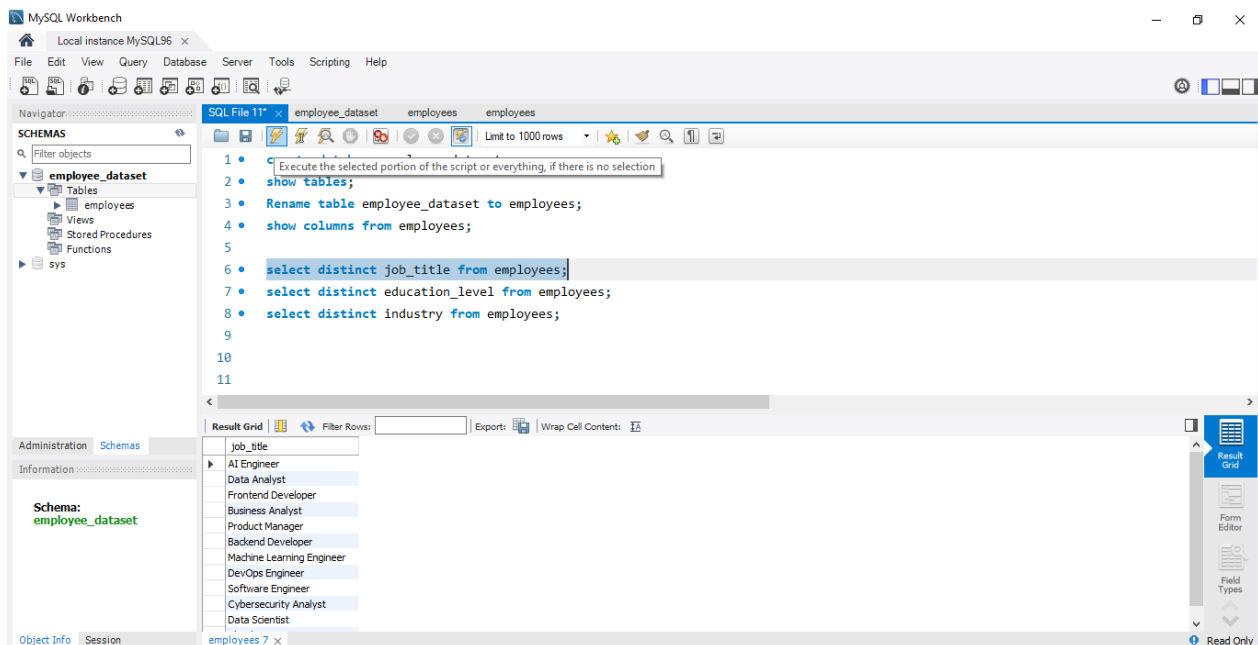
TASKS AND SOLUTIONS

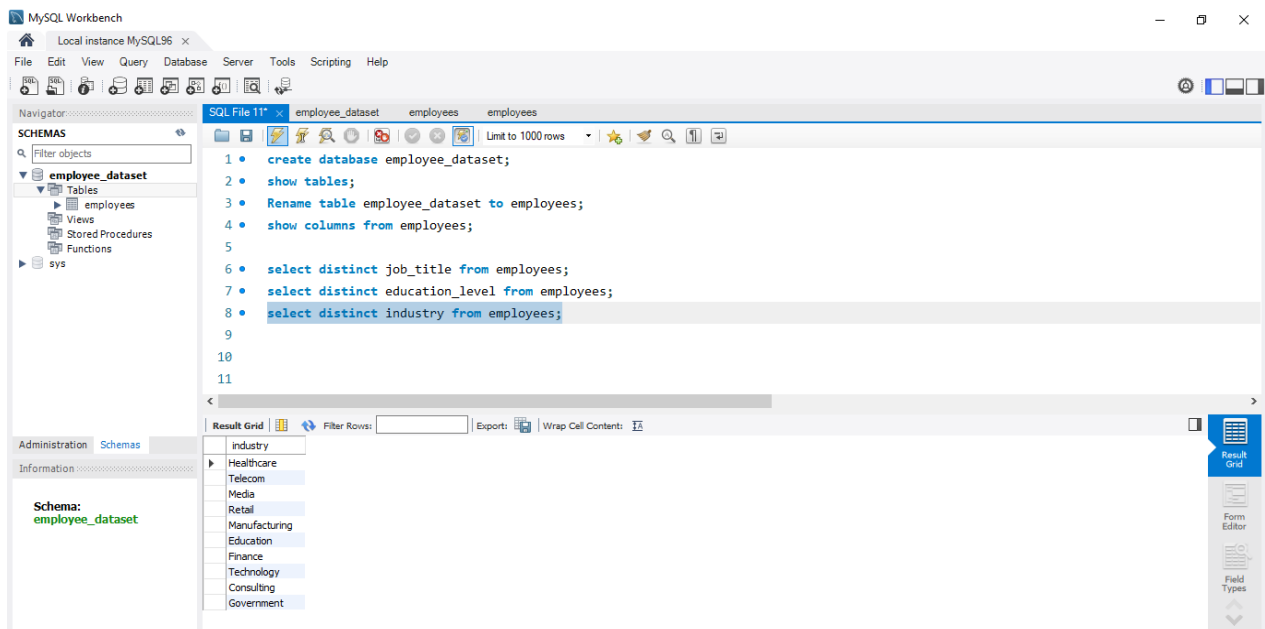
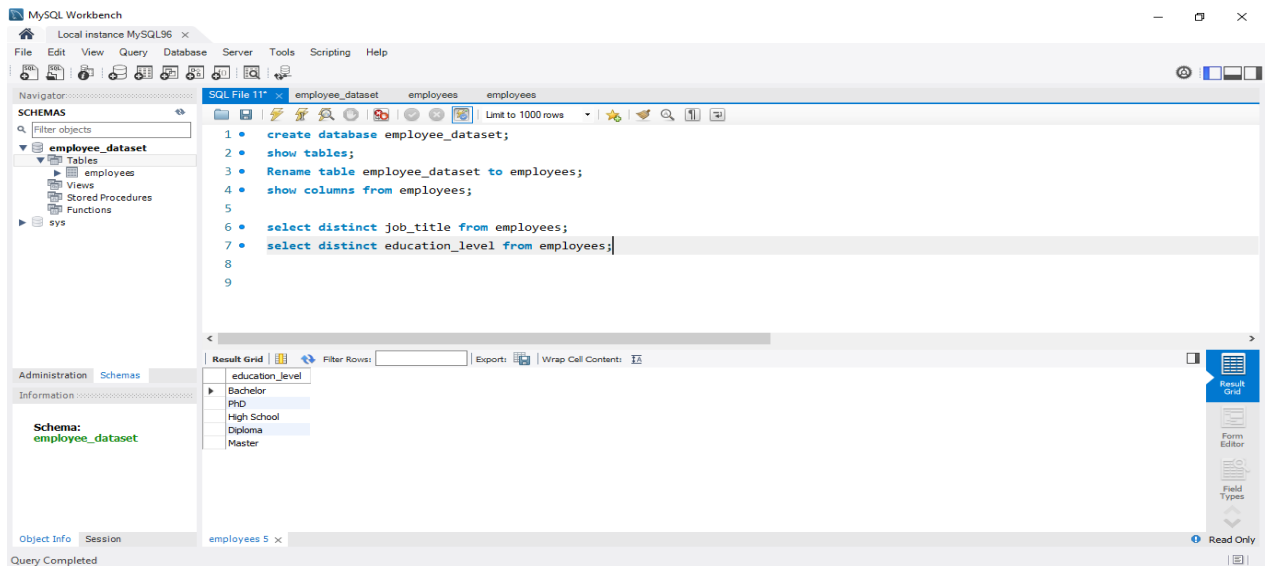
I created a database in MySQL Workbench and called it employee_dataset. I also created a table and named it employees which I use in performing the given task.

(1a) Distinct Values

This helps to identity all unique categories available in a field without repetition.

MySQL Query



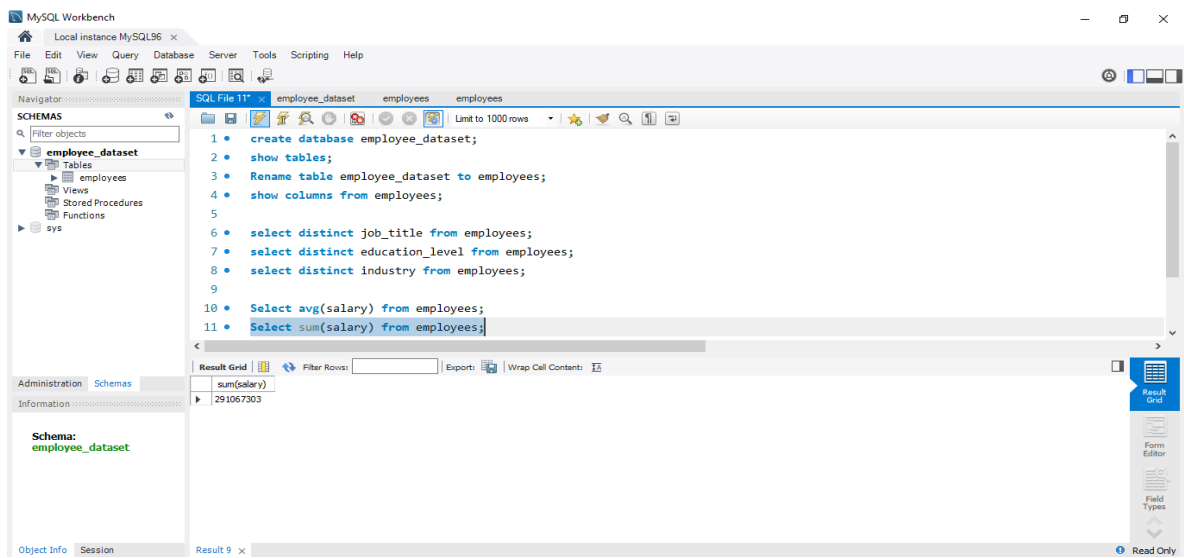
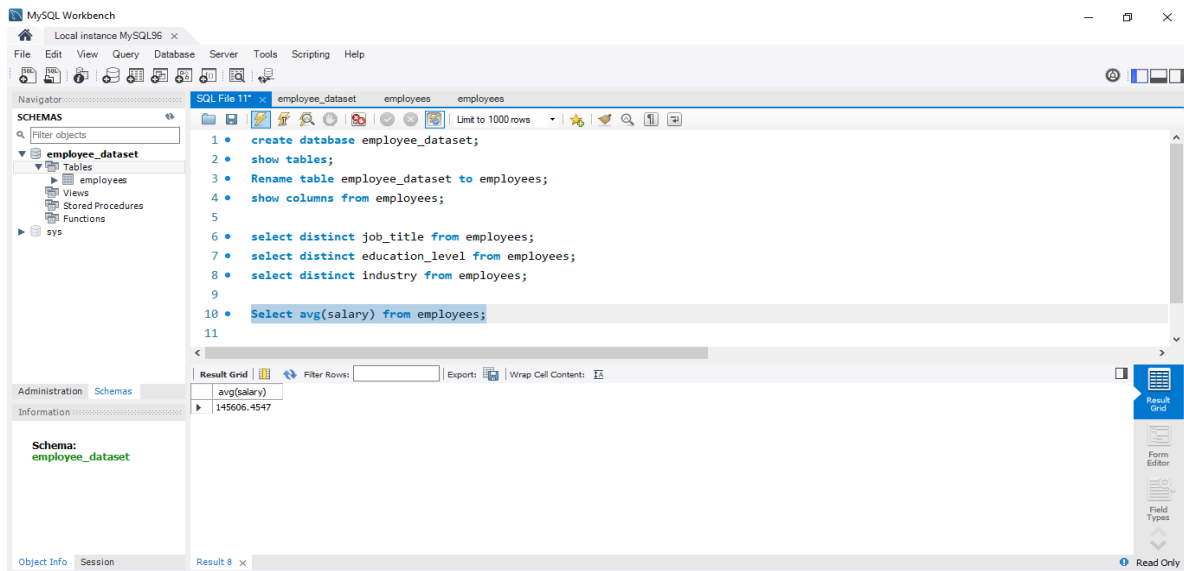


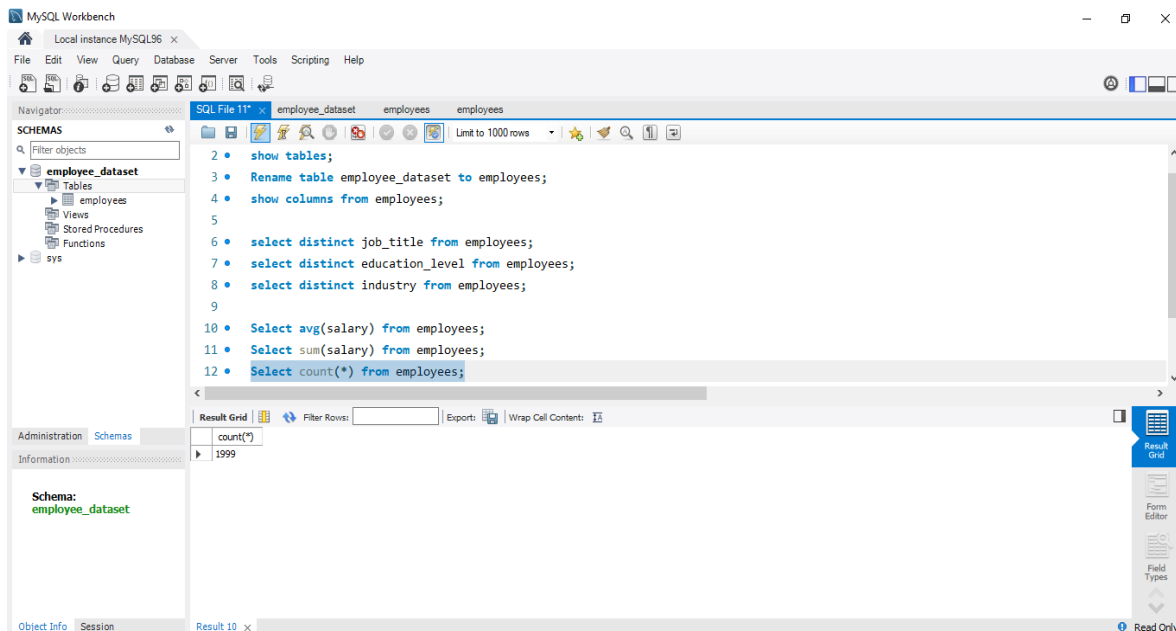
(2b) Create a function on the table.

I created a function that shows

- average salary
- sum of salary
- count of the total number of employees.

MySQL Query



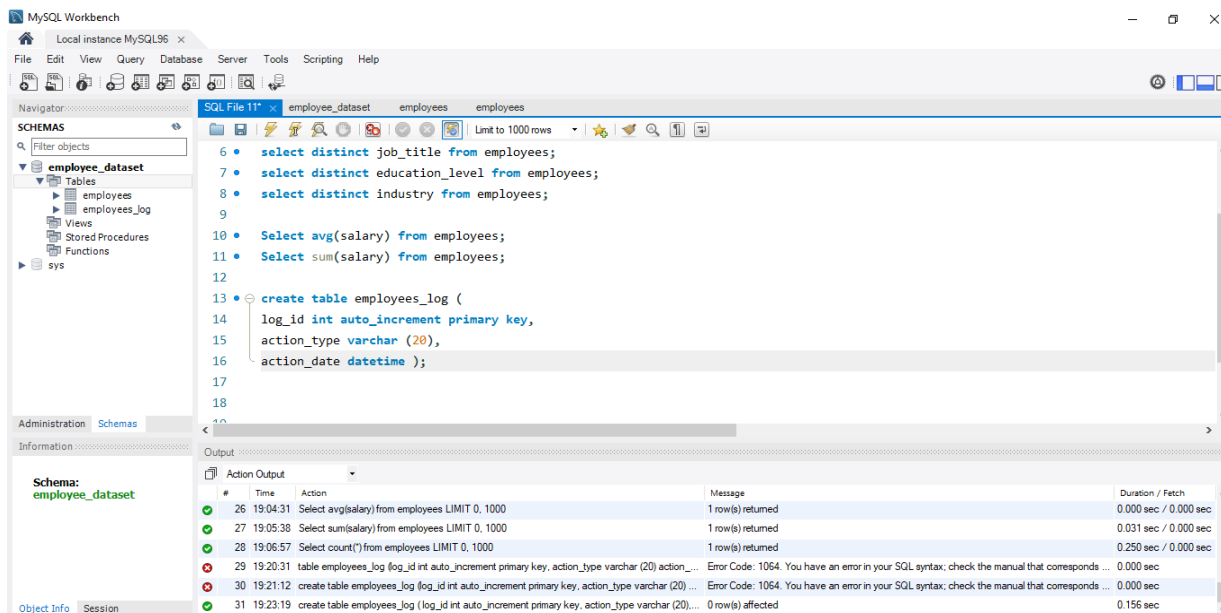


(1c) Create insert, delete and update triggers on your table.

A log table was created before the triggers to store the database activities. The INSERT, UPDATE and DELETE triggers automatically record all additions, modifications and deletions performed on the employees File table.

Create trigger

MySQL Query for the log table



Insert triggers

MySQL Query

The screenshot shows the MySQL Workbench interface with the 'employee_dataset' database selected. The SQL editor contains the following code:

```
15 action_type varchar(20),
16 action_date datetime);
17
18 DELIMITER //
19 CREATE TRIGGER trg_insert_employee
20 AFTER INSERT ON employees
21 FOR EACH ROW
22 BEGIN
23 INSERT INTO
24 employee_log(action_type, action_date)
25 VALUES ('INSERT', NOW());
26 END //
27 DELIMITER;
```

The Output tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
34	19:55:48	CREATE TRIGGER trg_insert_employee AFTER INSERT ON employees FOR EACH ROW ...	Error Code: 1359. Trigger already exists	0.016 sec
35	19:56:05	SELECT * FROM employee_dataset.employees_log LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
36	19:56:17	SELECT * FROM employee_dataset.employees_log LIMIT 0, 1000	1000 row(s) returned	0.047 sec / 0.016 sec
37	19:56:45	CREATE TRIGGER trg_insert_employee AFTER INSERT ON employees FOR EACH ROW ...	Error Code: 1359. Trigger already exists	0.000 sec
38	19:57:01	SELECT * FROM employee_dataset.employees_log LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
39	19:57:15	SELECT * FROM employee_dataset.employees_log LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec

UPDATE TRIGGER

MySQL Query

The screenshot shows the MySQL Workbench interface with the 'employee_dataset' database selected. The SQL editor contains the following code:

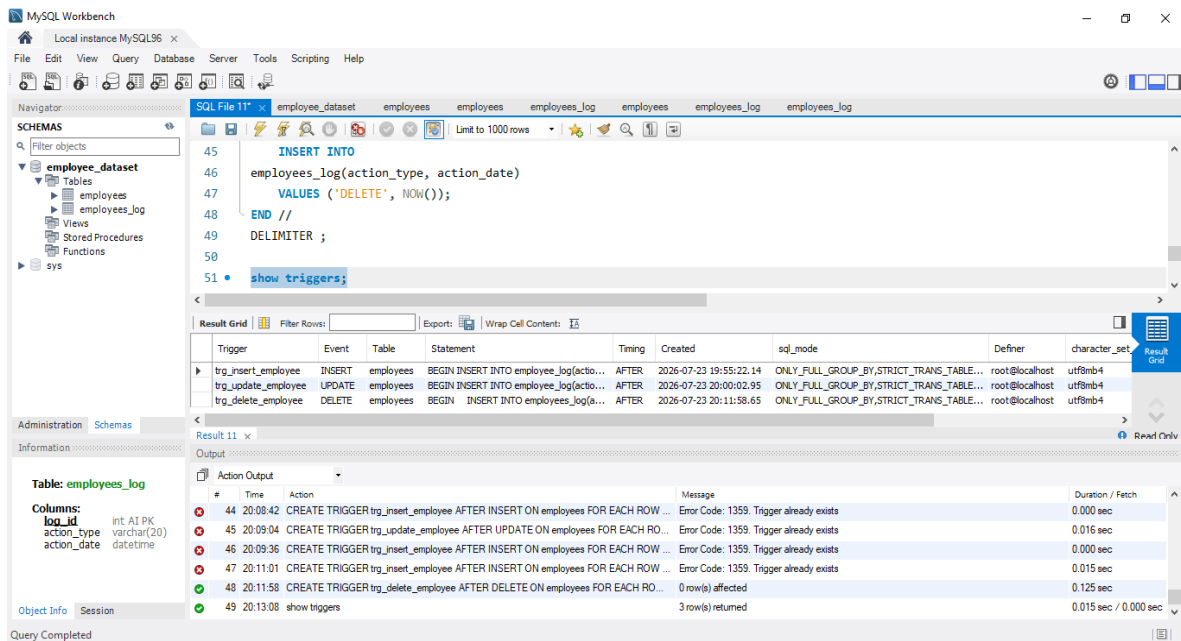
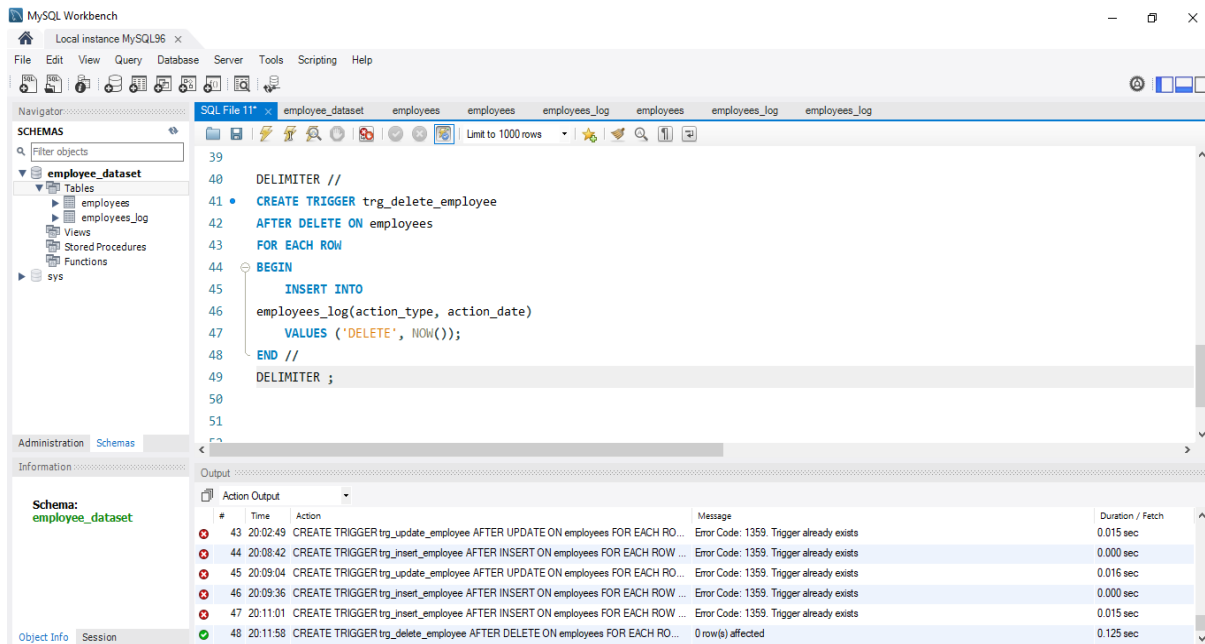
```
27 DELIMITER ;
28
29 DELIMITER //
30 CREATE TRIGGER trg_update_employee
31 AFTER UPDATE ON employees
32 FOR EACH ROW
33 BEGIN
34 INSERT INTO
35 employee_log(action_type, action_date)
36 VALUES ('UPDATE', NOW());
37 END //
38 DELIMITER ;
```

The Output tab shows the execution results:

#	Time	Action	Message	Duration / Fetch
38	19:57:01	SELECT * FROM employee_dataset.employees_log LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
39	19:57:15	SELECT * FROM employee_dataset.employees_log LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
40	20:00:02	CREATE TRIGGER trg_update_employee AFTER UPDATE ON employees FOR EACH RO...	0 row(s) affected	0.047 sec
41	20:00:02	DELIMITER;	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds ...	0.000 sec
42	20:02:07	CREATE TRIGGER trg_insert_employee AFTER INSERT ON employees FOR EACH ROW ...	Error Code: 1359. Trigger already exists	0.000 sec
43	20:02:49	CREATE TRIGGER trg_update_employee AFTER UPDATE ON employees FOR EACH RO...	Error Code: 1359. Trigger already exists	0.015 sec

DELETE TRIGGER

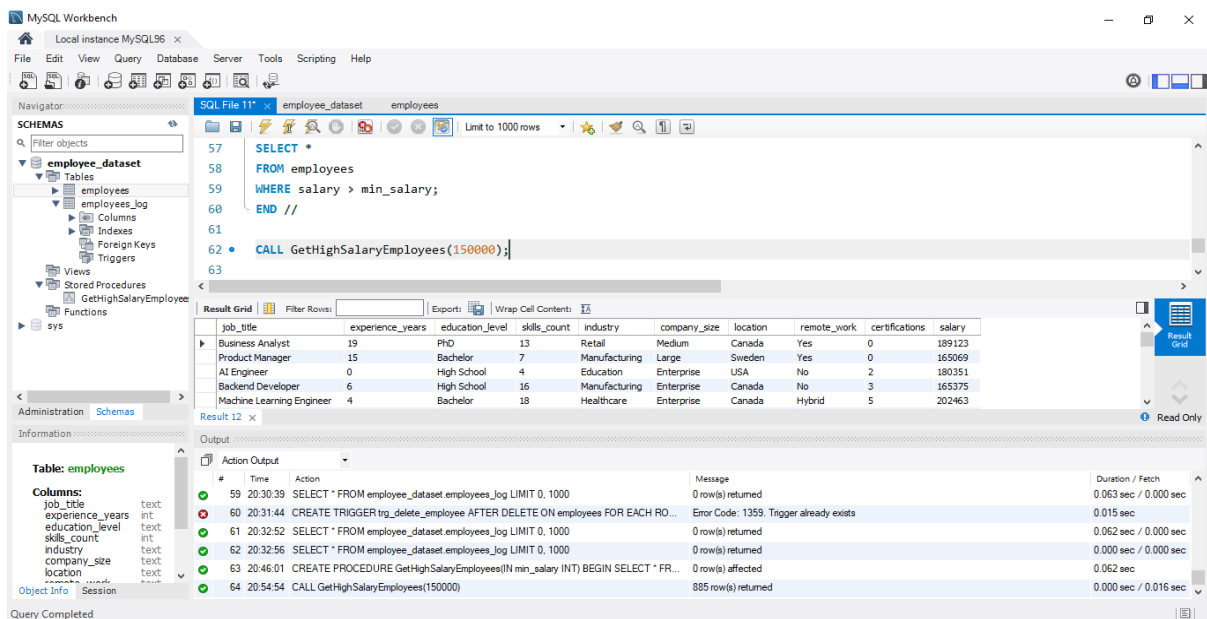
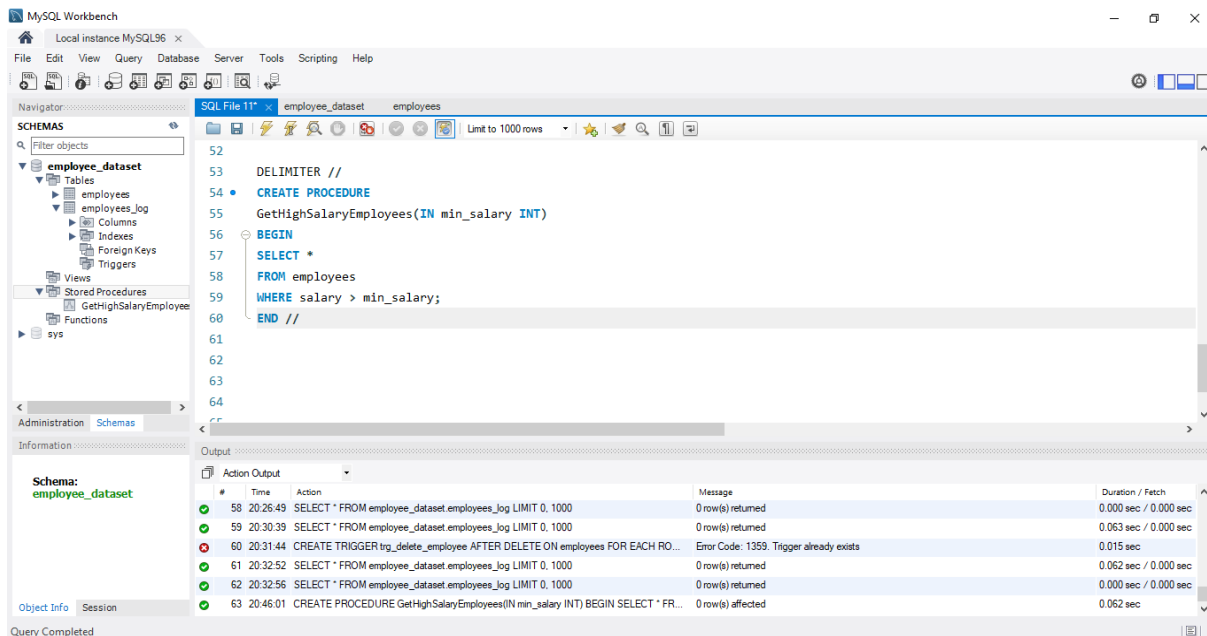
MySQL Query



(1d) Create stored procedure

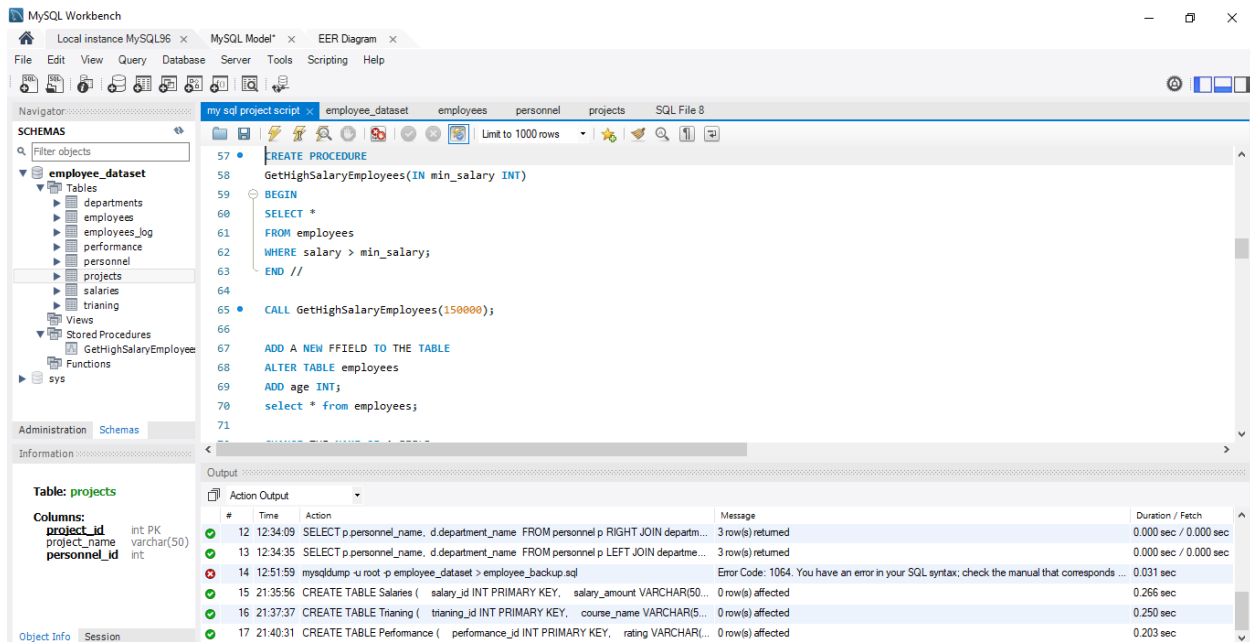
This procedure display employees whose salary is greater than the value passed into the procedure.

MySQL Query



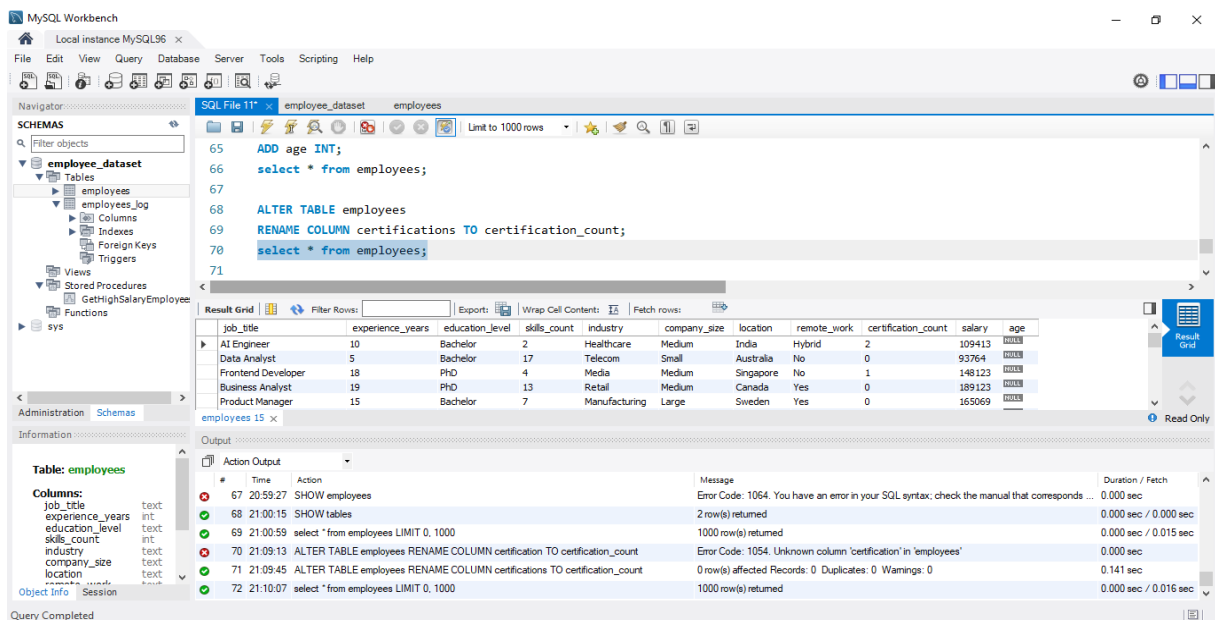
(1e) Add a new field to the table
Adding age to the table.

SQL Query



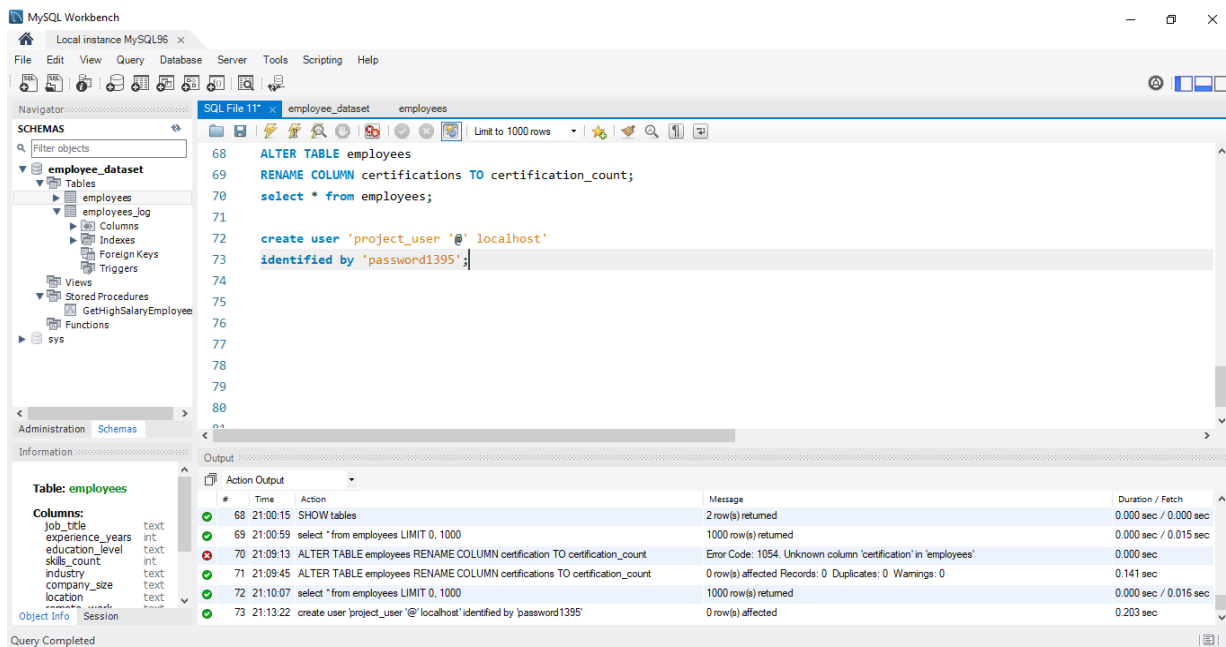
(1f) Change the name of a field
Renaming certification to certification_count

SQL Query



(1g) Create a new user
This creates a new database account.

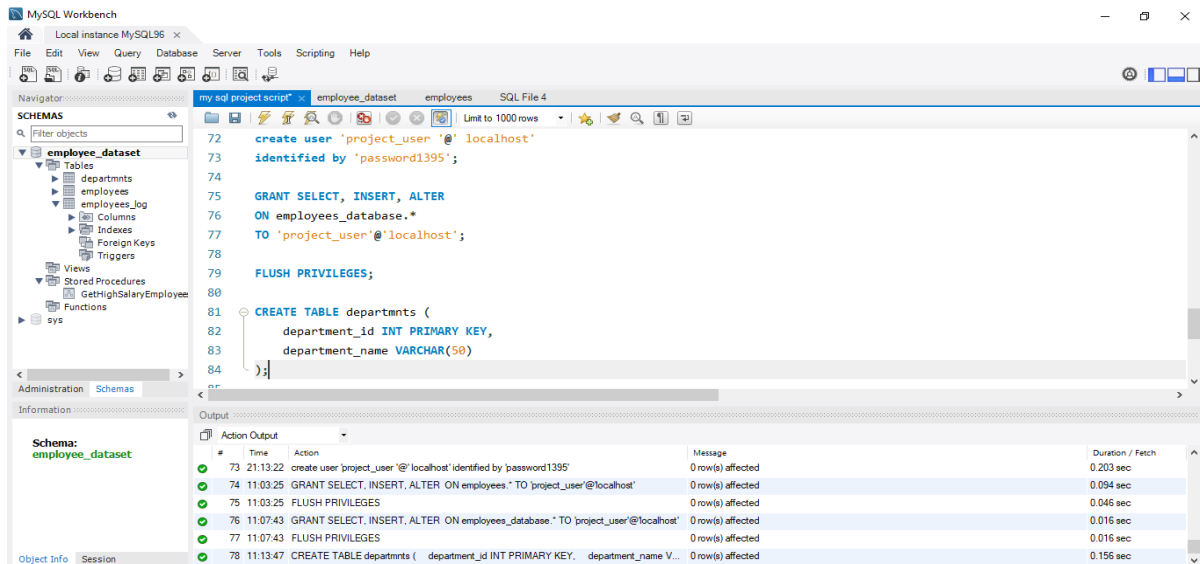
SQL Query



(1h) Grant Privileges

This helps to control what users are allowed to do in the database. Select can view data, insert can add records, alter can modify table structure.

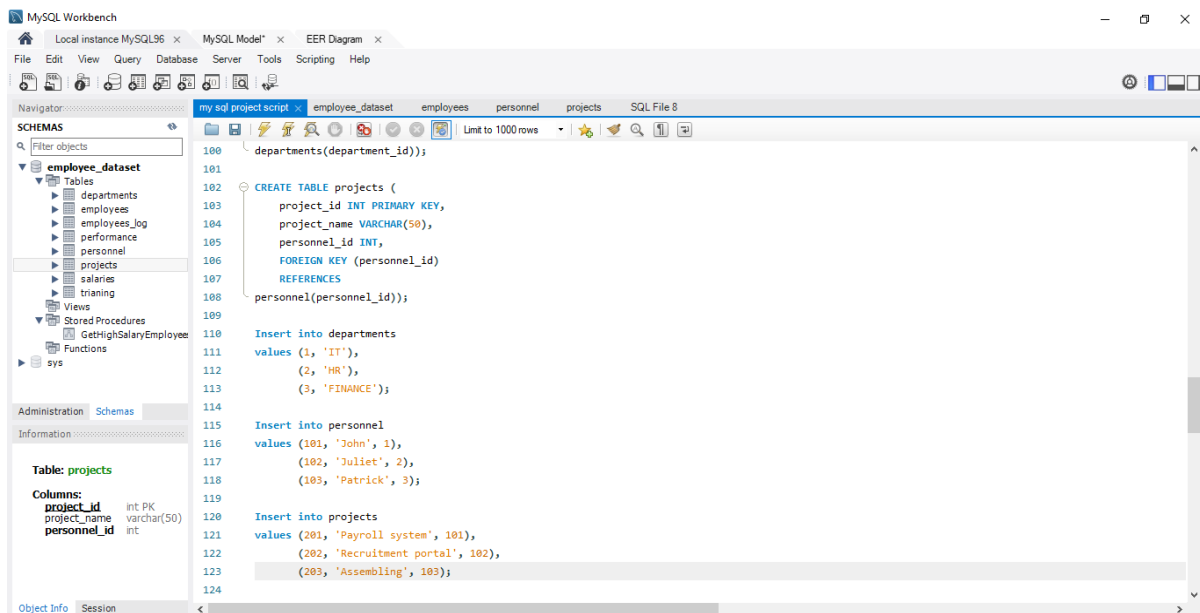
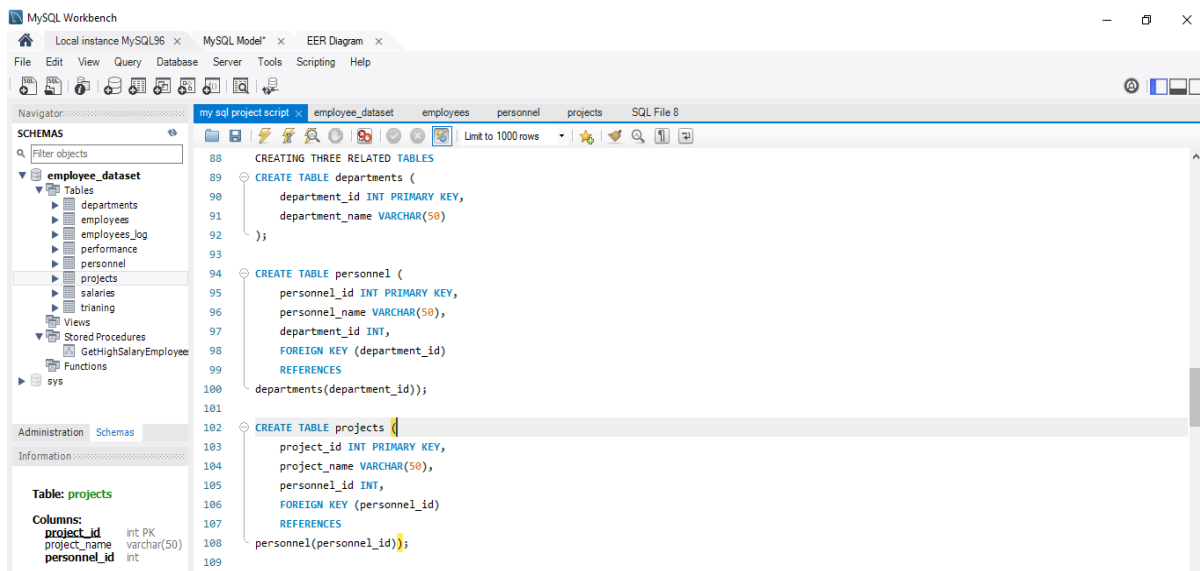
SQL Query



(1i) Create three related tables

I created department table, personnel table and project table and inserted records into the table so as to be able to show the join.

SQL Query



INNER JOIN

This returns only matching records. It shows only records that have matching values in both tables.

MY SQL Query

MySQL Workbench interface showing a query execution. The query is:

```

115 (203, 'Assembling', 103);
116
117 SELECT p.personnel_name,
118        d.department_name
119 FROM personnel p
120 INNER JOIN departments d
121 ON p.department_id = d.department_id;
122

```

The result grid shows the following data:

personnel_name	department_name
John	IT
Juliet	HR
Patrick	FINANCE

The output pane shows the execution log with a message: 'Error Code: 1054. Unknown column 'd' in field list'.

LEFT JOIN

This returns all records from the left table and matching records from the right table.

MY SQL Query

MySQL Workbench interface showing a query execution. The query is:

```

121 ON p.department_id = d.department_id;
122
123 SELECT p.personnel_name,
124        d.department_name
125 FROM personnel p
126 LEFT JOIN departments d
127 ON p.department_id = d.department_id;
128

```

The result grid shows the following data:

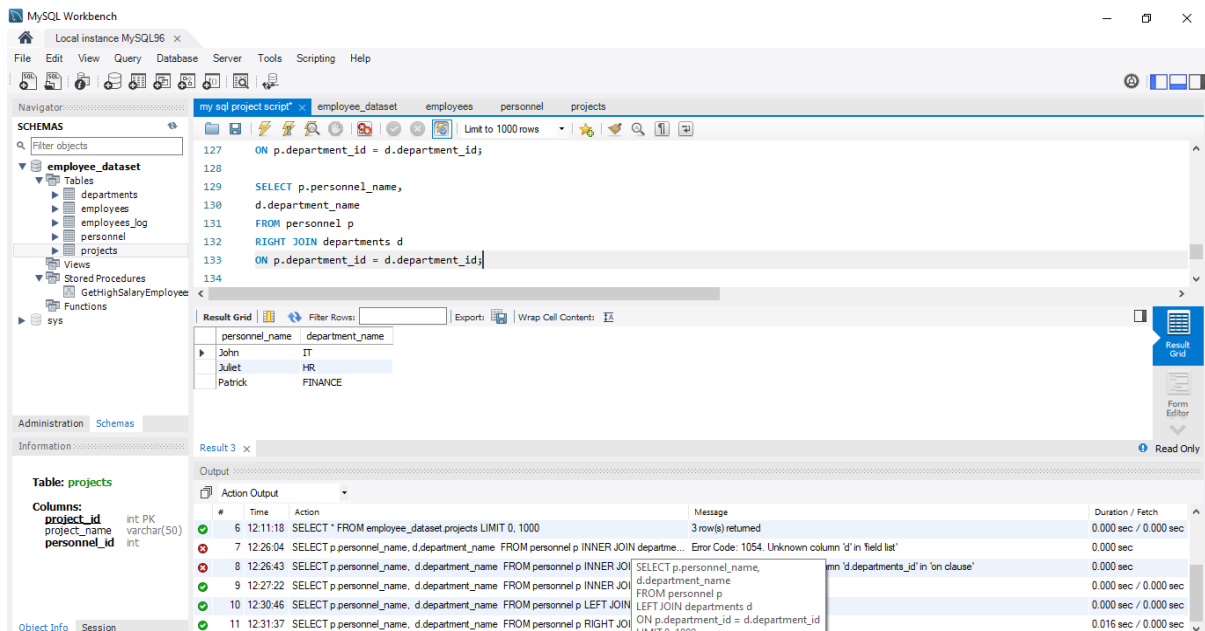
personnel_name	department_name
John	IT
Juliet	HR
Patrick	FINANCE

The output pane shows the execution log with a message: 'Error Code: 1054. Unknown column 'd' in field list'.

RIGHT JOIN

This returns all records from the right table and matching records from the left table.

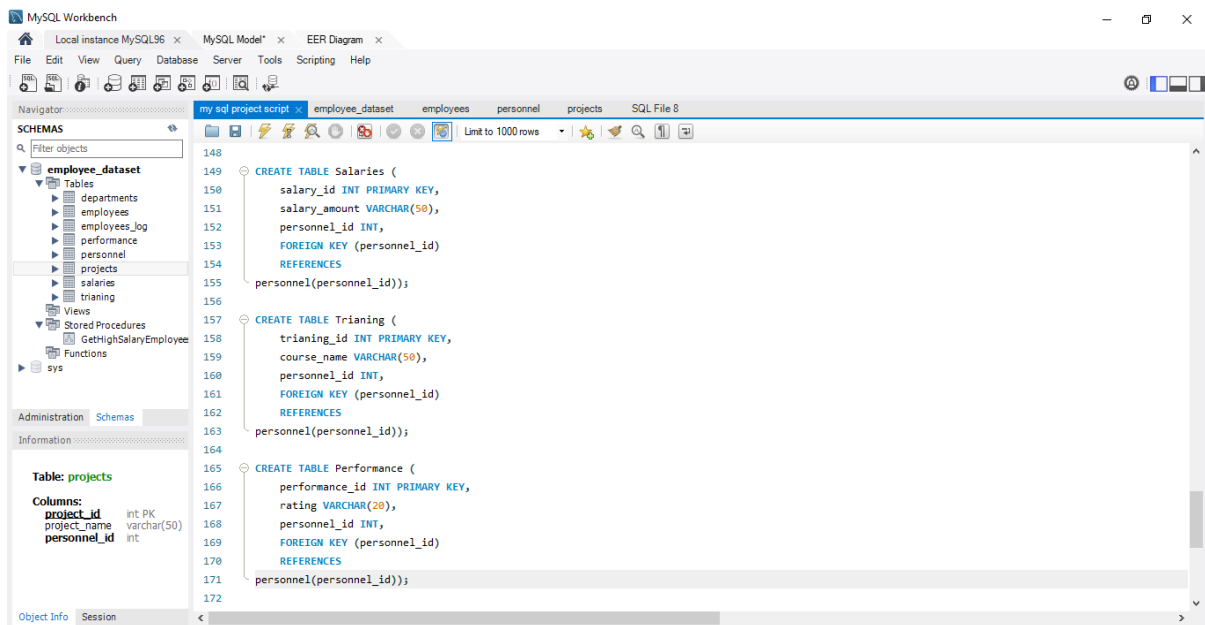
MY SQL Query



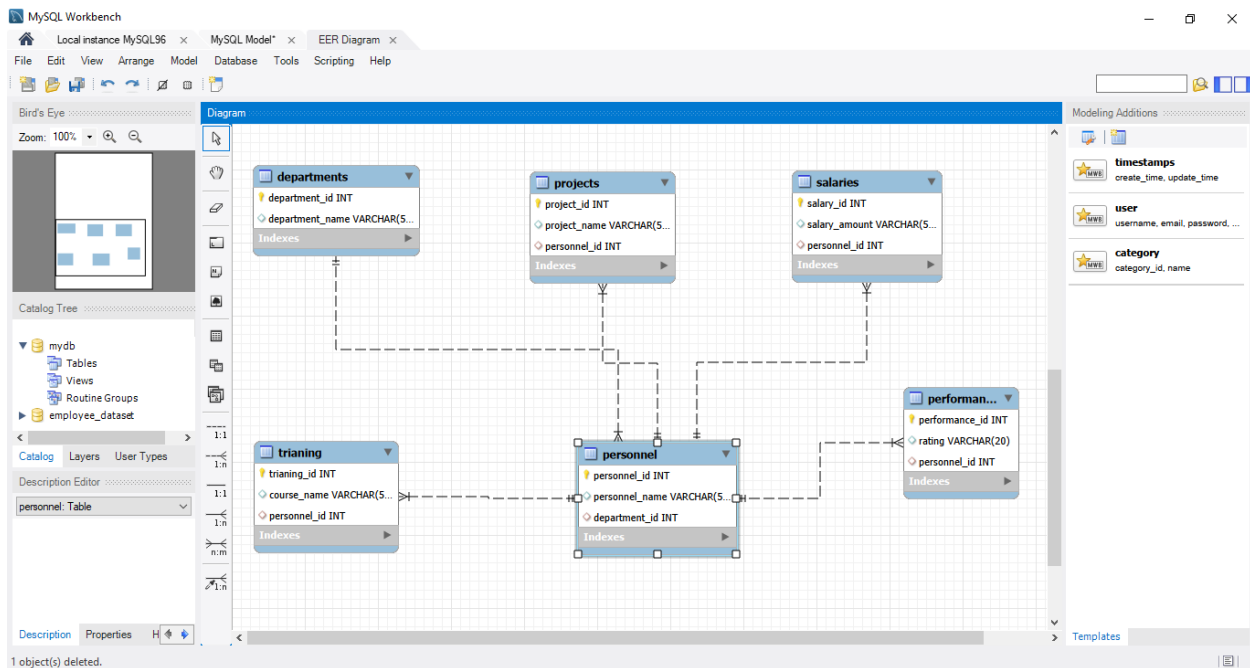
(2c) Create ERD with 6 related tables.

I have three tables created already so I created three more tables salaries, training and performance.

MySQL Query



I created six related tables in MySQL and generated the ERD using MySQL workbench. The tables are department, personnel, projects, salaries, training and performance.



The relationships are all one to many.

Department(1:M) Personnel

Personnel(1:M) Projects

Personnel(1:M) Salaries

Personnel(1:M) Training

Personnel(1:M) Performance

MYSQL THEORY ANSWERS

(2a) Backup and Restore Database

These are command-line commands

Backup

```
Mysqldump -u root -p employee_dataset > employee_backup.sql
```

Restore

```
Mysql -u root -p employee_dataset < employee_backup.sql
```

(b). Difference between char and varchar

1. Char stores fixed-length data and pads unused space with blanks while varchar stores variable-length and uses only the space needed for actual text.
2. Char always keeps exact size you set. While varchar changes size based on your text.

(d). Differences between SQL and MYSQL

1. SQL is standard query language while MYSQL is data management system.
2. SQL is used to manipulate data while MYSQL is uses SQL to manage data.
3. SQL is standard language while MYSQL is software product developed by oracle.

(e). Why are triggers created

Triggers are created

1. To automatically perform actions after database events.
2. To maintain audit logs
3. To enforce business rules.
4. To improve data integrity.
5. To Track inserts, updates and deletes.

(f). Difference between primary key and foreign key

1. Primary key uniquely identifies records while foreign key links tables together.
2. Primary key cannot be null while foreign key can contain null values.
3. Primary key allows one primary key per table while foreign key allows multiple foreign keys.

(g). Differences between DBMS and RDBMS

1. DBMS stores data as files while RDBMS stores data as table.
2. No relationship is required in DBMS While relationship is required in RDBMS.
3. DBMS IS less secure while RDBMS is more secure,
4. Examples of DBMS is XML, File system while examples of RDBMS are MySQL, Oracle, PostgreSQL

CONCLUSION

This project demonstrates practical database management using MySQL. Various SQL concepts such as triggers, stored procedures, joins and database security were successfully implemented.